

CSCI 361 Lecture 3:

Regular Languages

Shikha Singh

Announcements & Logistics

- **Assignment 2** released, due Sept 17 (Wed) at 10 pm
- Hand in Exercise #2, pick up Exercise #3
- **Questions?**

Last Time

- Definitions of DFA and regular languages
- Practice with mapping DFAs to language recognized and vice versa
- Closure of regular languages under complement, union and intersection

Today

- Introduce a nondeterministic finite automaton: NFA
- Practice with NFAs
- Equivalence theorem: DFA $\iff$ NFA

Concatenation

- Let A and B be languages over Σ .
- **Definition.** Concatenation of A and B , denoted $A \circ B$ is defined as

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

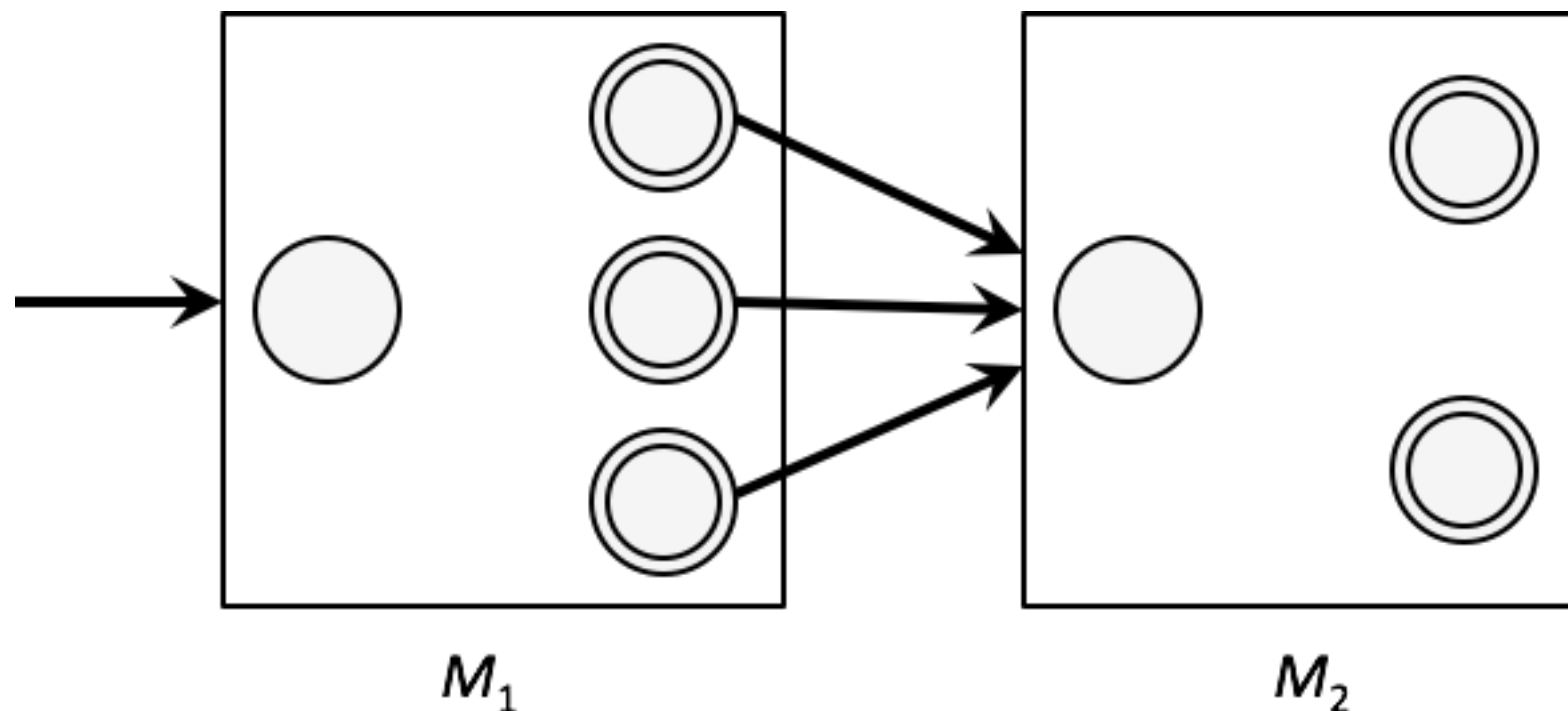
- **Question.** Are regular languages closed under concatenation?

Intuition: Closed Under Concatenation

- Let A and B be languages over Σ .
- **Definition.** Concatenation of A and B , denoted $A \circ B$ is defined as

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

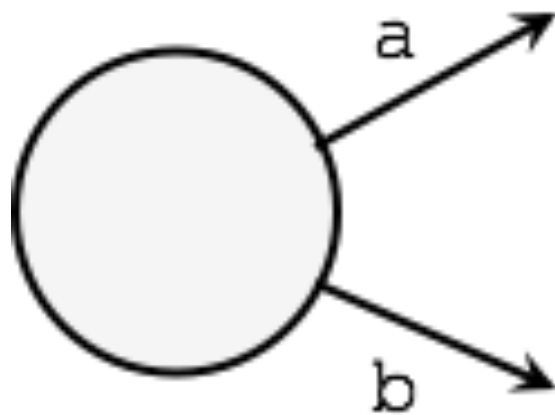
- **Question.** Are regular languages closed under concatenation?



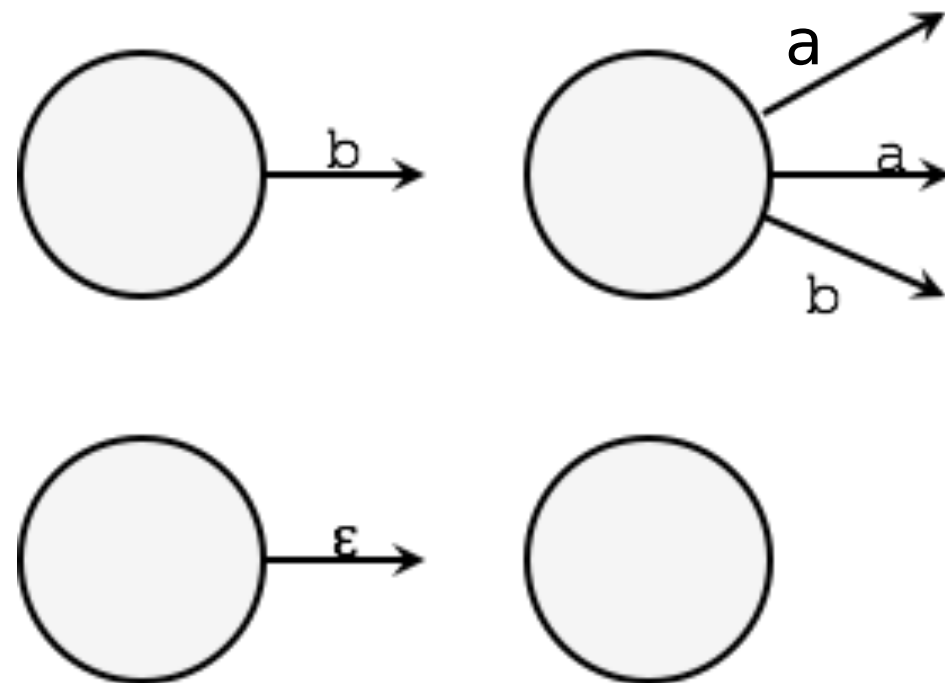
Non-deterministic Finite Automaton (NFA)

Relaxing the Rules

- Deterministic Finite Automaton (**DFA**)

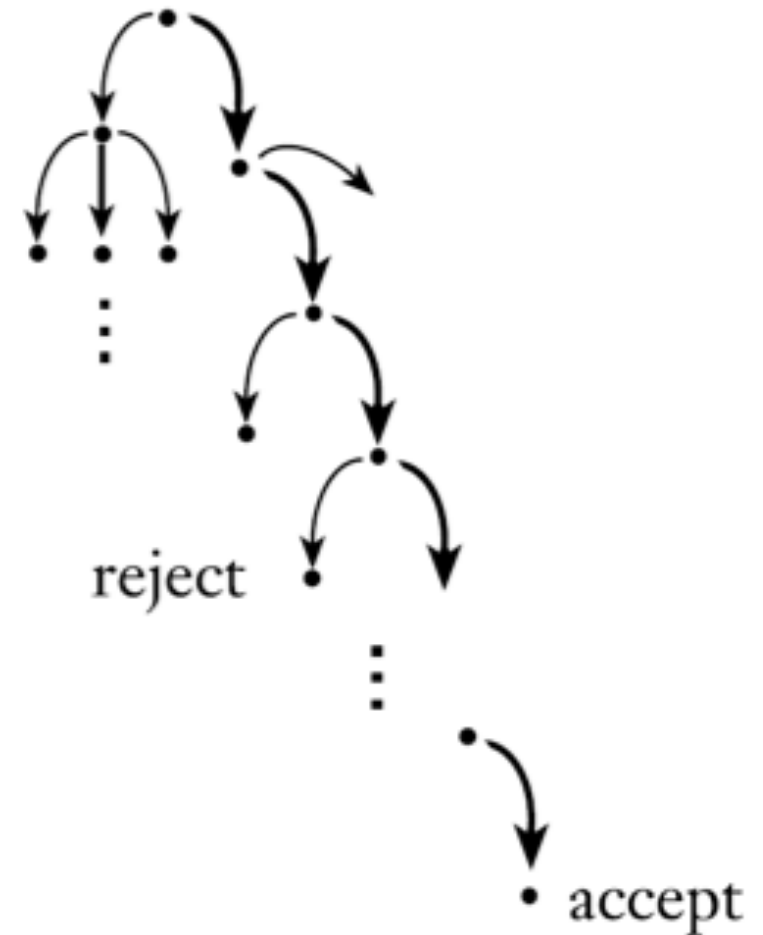
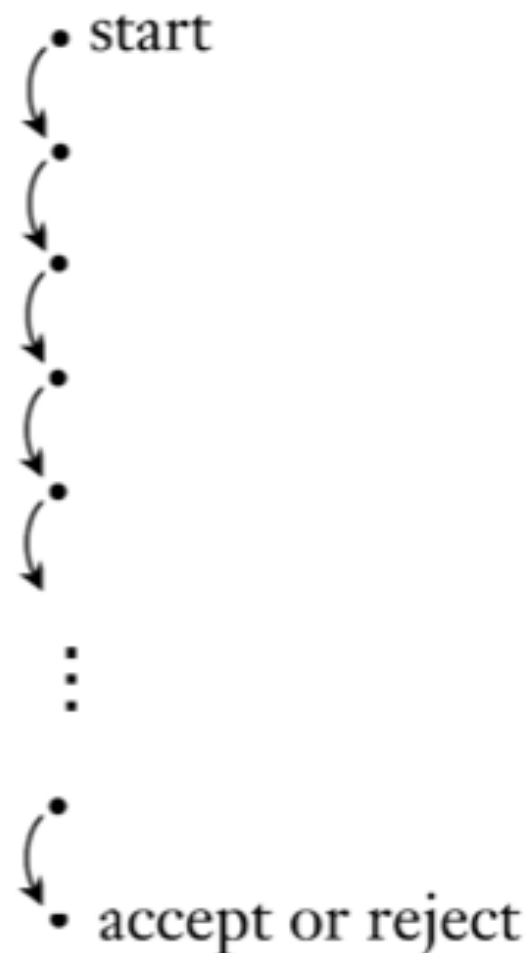


- Non-deterministic Finite Automaton (**NFA**)

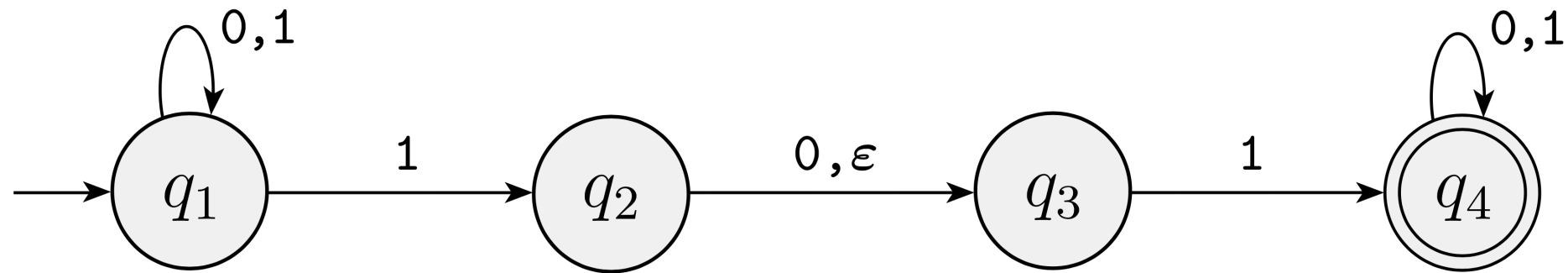


How Does Computation Proceed?

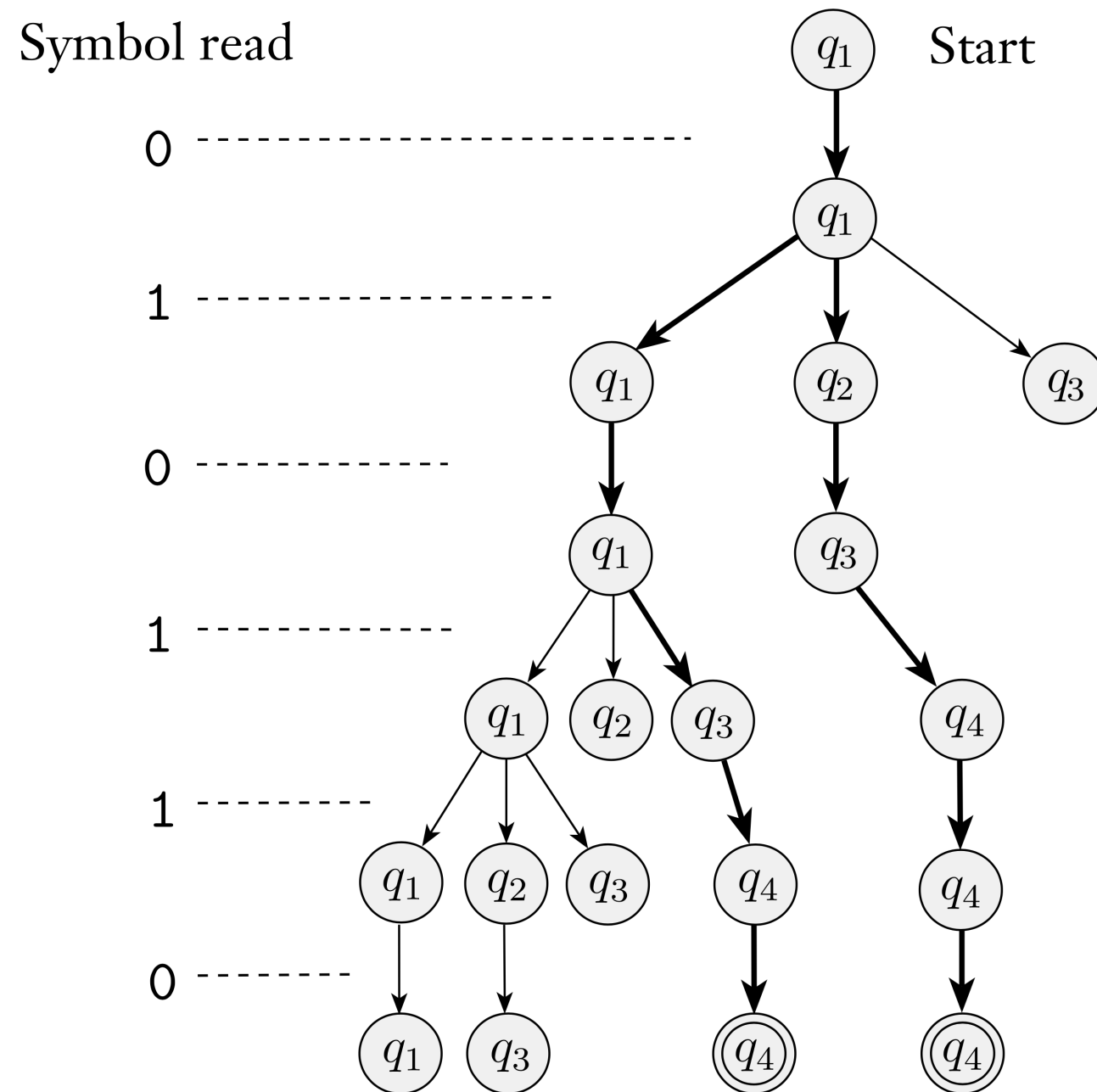
- Deterministic Finite Automaton (**DFA**)
- Non-deterministic Finite Automaton (**NFA**)



Example of NFA N_1 from Sipser



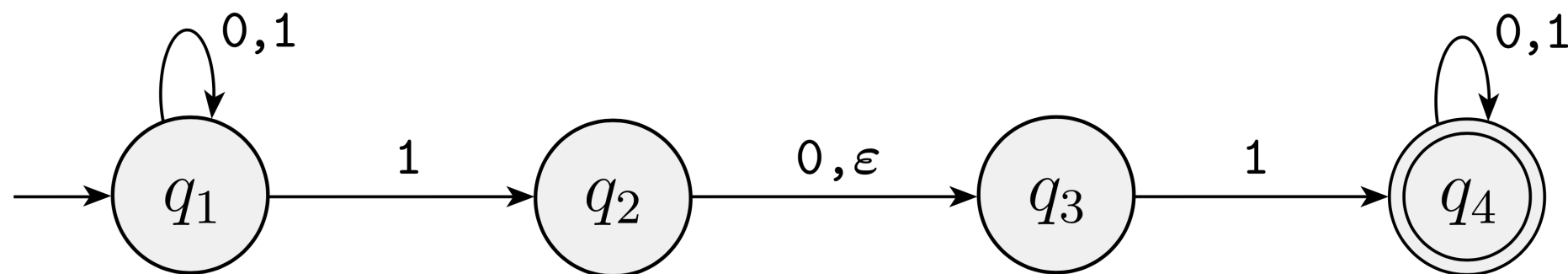
Input: 010110



Formal Definition: NFA

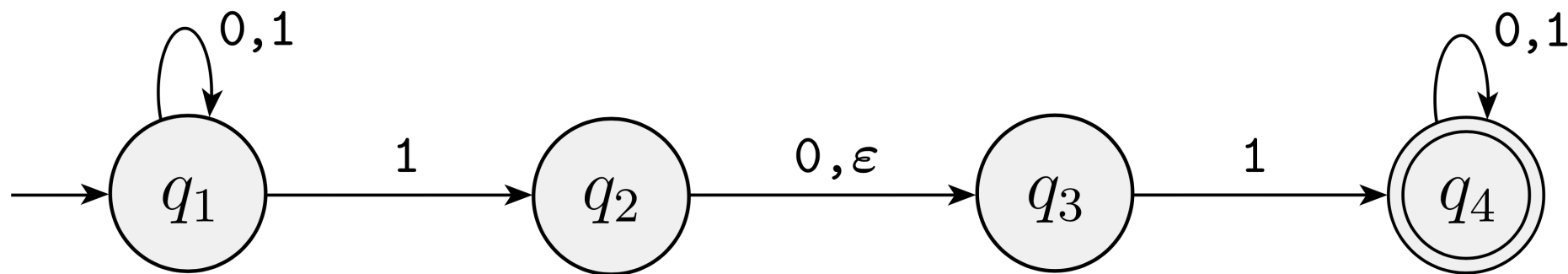
A non-deterministic finite automaton is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ is the transition function,
- $q_o \in Q$ is the **start** state and $F \subseteq Q$ is the set of **accept** states.



NFA Computation

- Let $N = (Q, \Sigma, \delta, q_0, F)$ be a non-deterministic finite automaton and let $w = w_1w_2\cdots w_n$ be a string where each $w_i \in \Sigma$. Then N **accepts** w if there is a sequence of $r_0, r_1, \dots, r_n$ in Q such that
 - $r_0 = q_0$
 - $r_{i+1} \in \delta(r_i, w_{i+1})$ for $i = 0, 1, \dots, n - 1$ and
 - $r_n \in F$

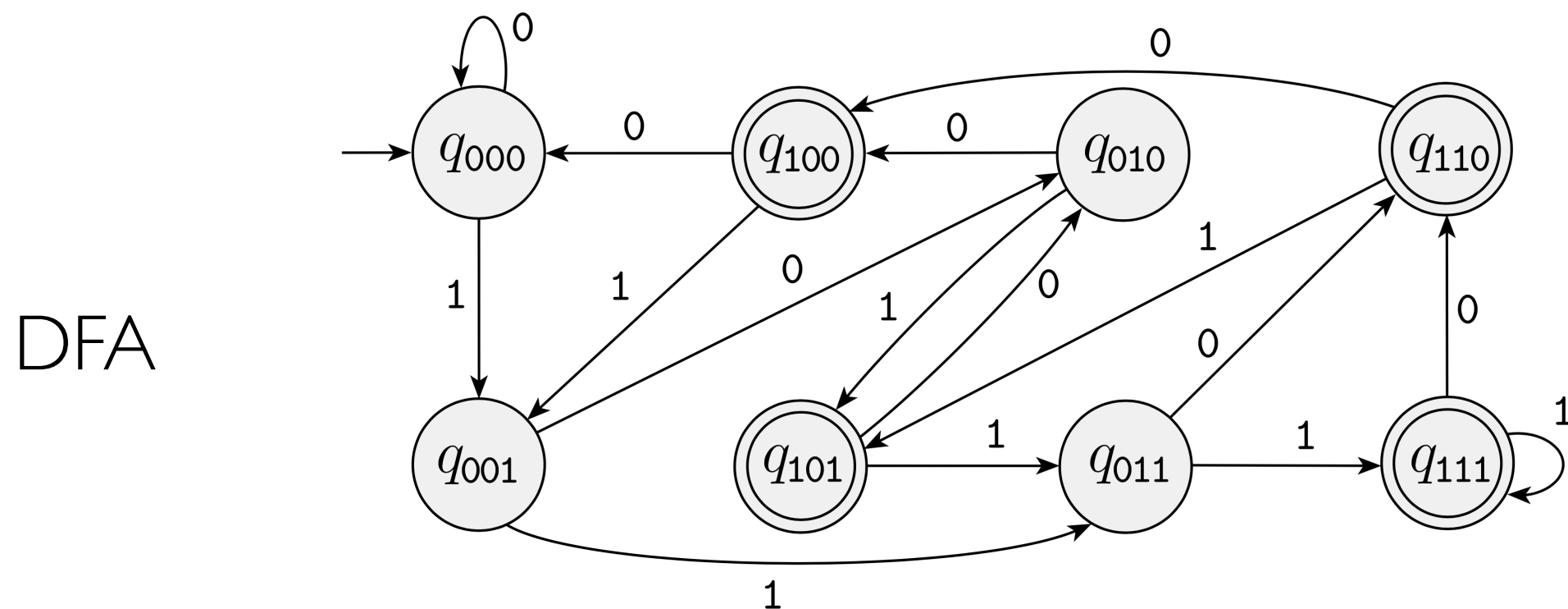
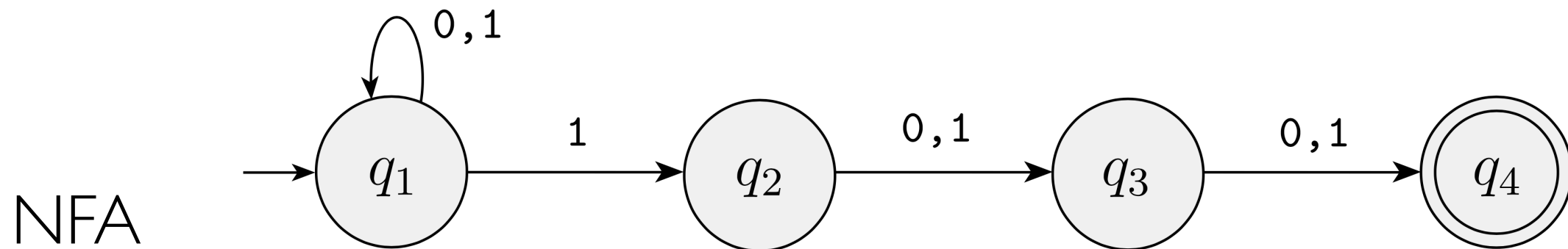


Class Exercise

- Build an NFAs to recognize the following languages:
- $L_1 = \{w \mid w \in \{0,1\}^* \text{ and has a 1 in the 3rd position from the end}\}$
- $L_2 = \{w \mid w \in \{0,1\}^* \text{ and } w \text{ begins with } 010 \text{ or ending with } 110\}$

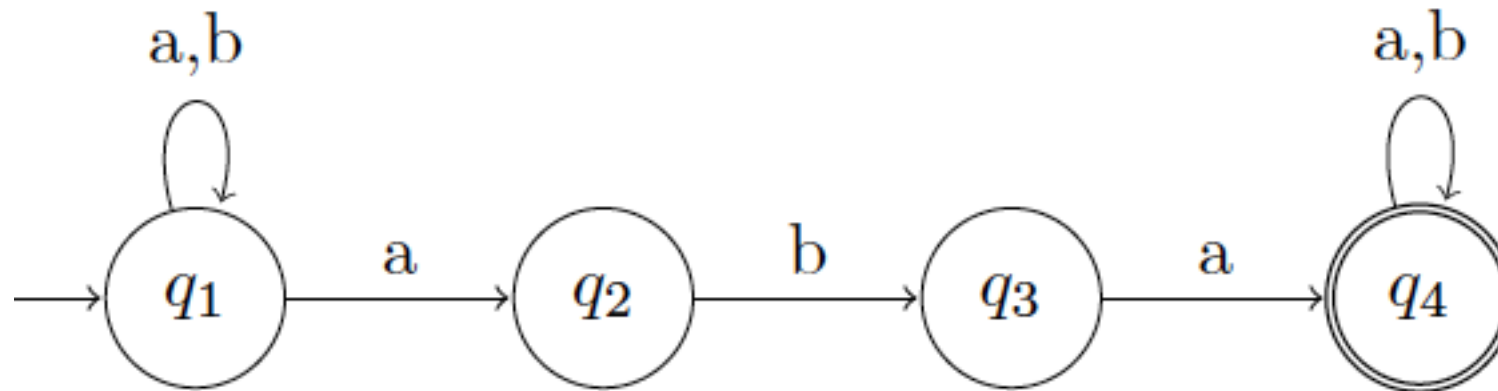
Nondeterminism is Your Friend

- Build an NFA to recognize the following language:
- $L = \{w \mid w \in \{0,1\}^* \text{ and has a 1 in the 3rd position from the end}\}$



More Examples

- What is the languages recognized by this NFAs?



DFA $\iff$ NFA
Equivalence

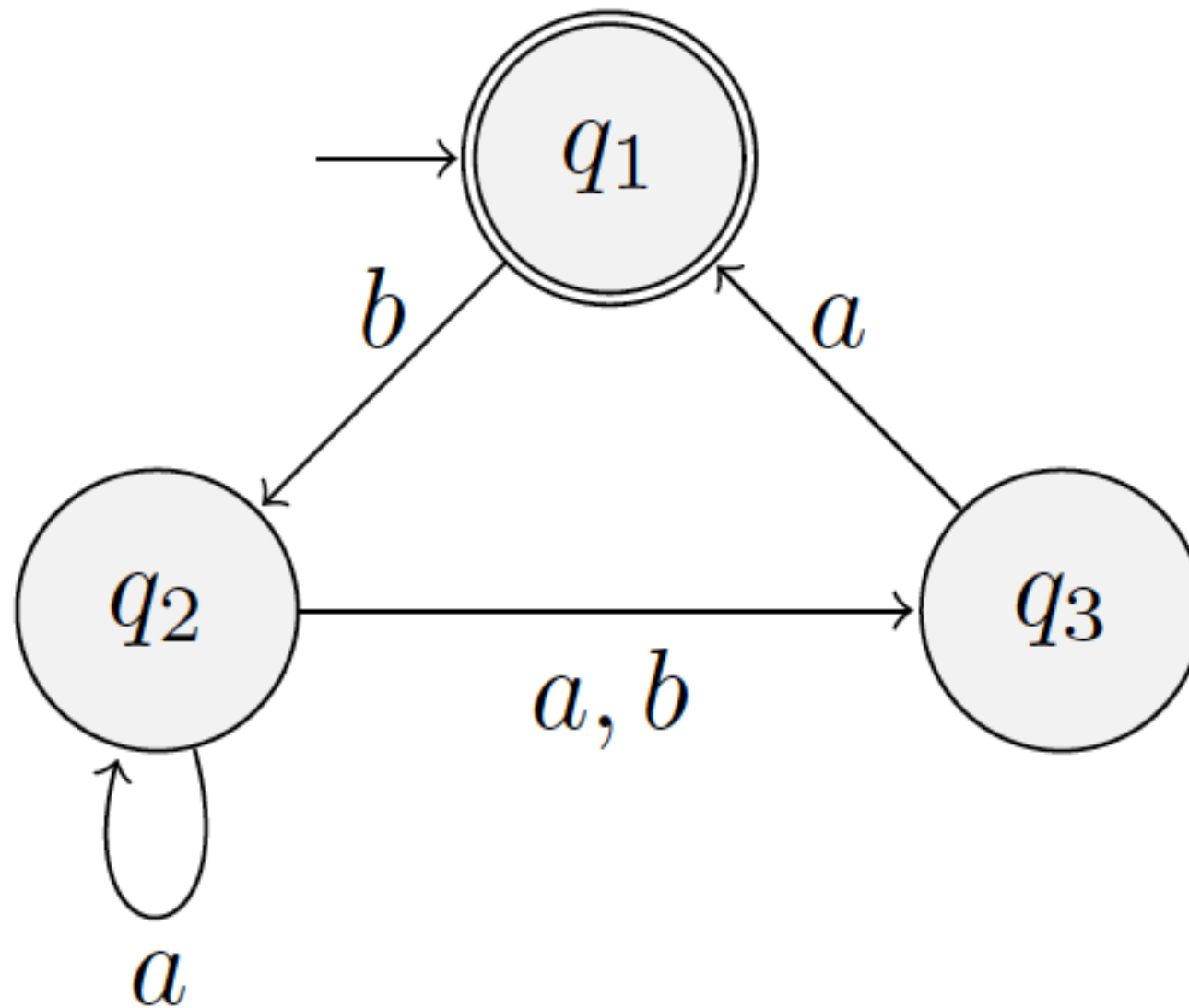
Equivalence

- **Definition.** Two machines are equivalent if they recognize the same language.
- **Theorem.** Given any NFA N there exists an equivalent DFA M and vice versa.
 - One direction is easy: every DFA is also an NFA by definition.
 - Need to show can construct a DFA M such that $L(M) = L(N)$

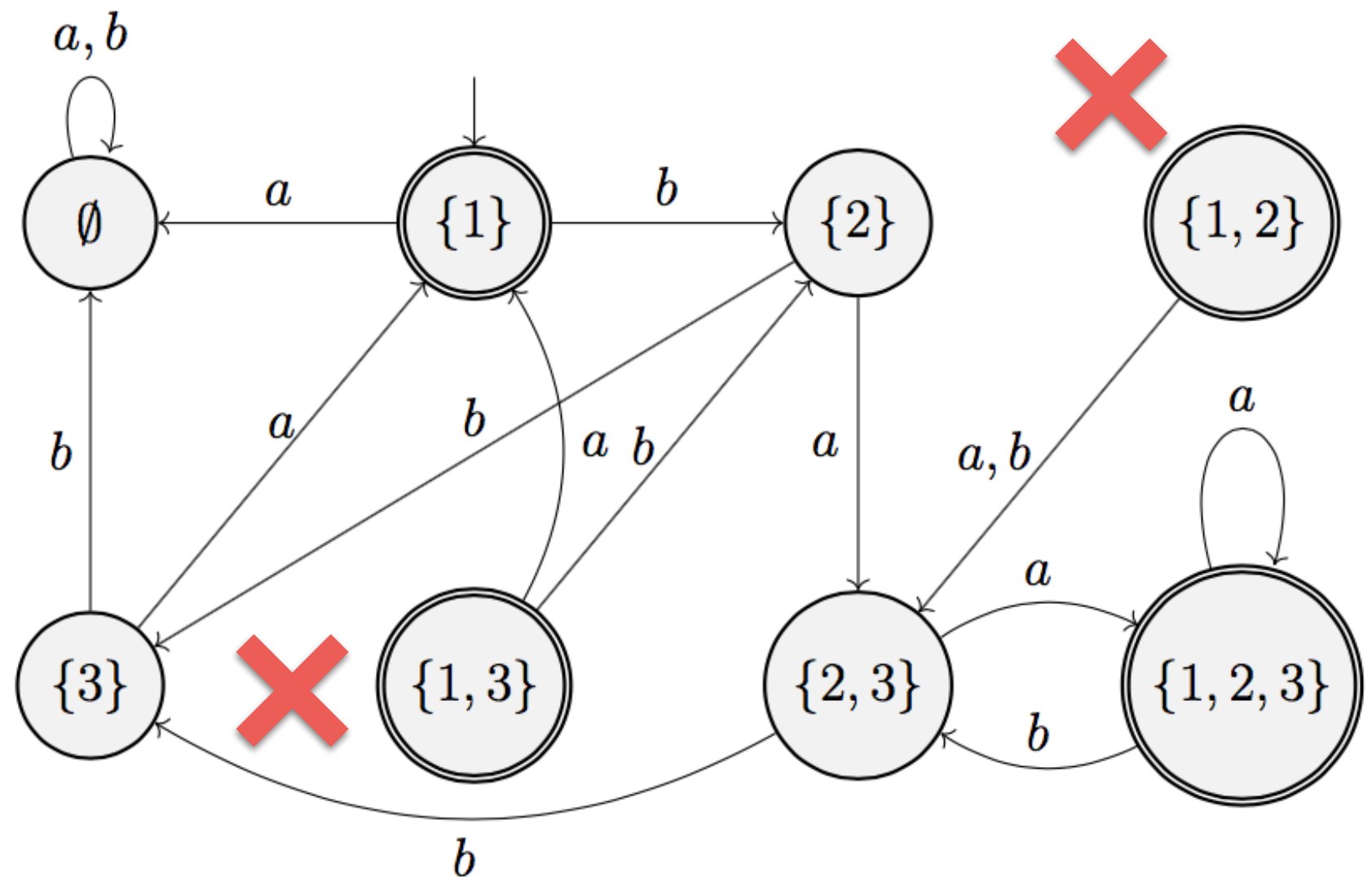
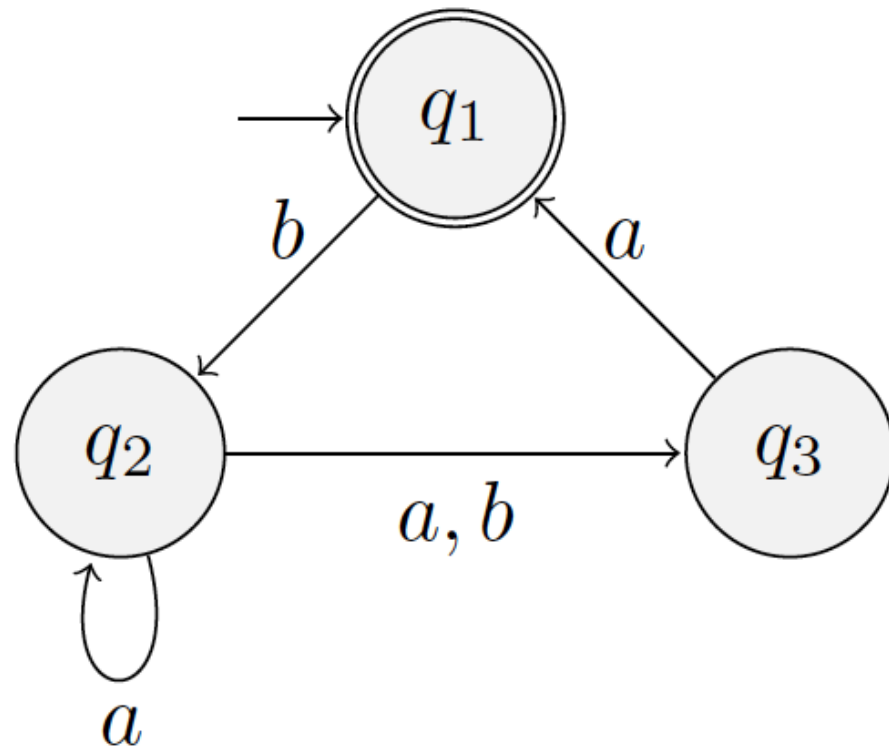
Creating an Equivalent DFA

- **Theorem.** Given any NFA $N = (Q, \Sigma, \delta, q, F)$ there exists an equivalent DFA M .
- **Proof outline:** M "simulates" N by having a larger state space
 - If N has k states, M will have 2^k states to account for any possible subset of N 's states
- In particular, $Q_M = \mathcal{P}(Q)$
- First, let's ignore ϵ transitions
- How can M simulate N ?

Example: Equivalent DFA?



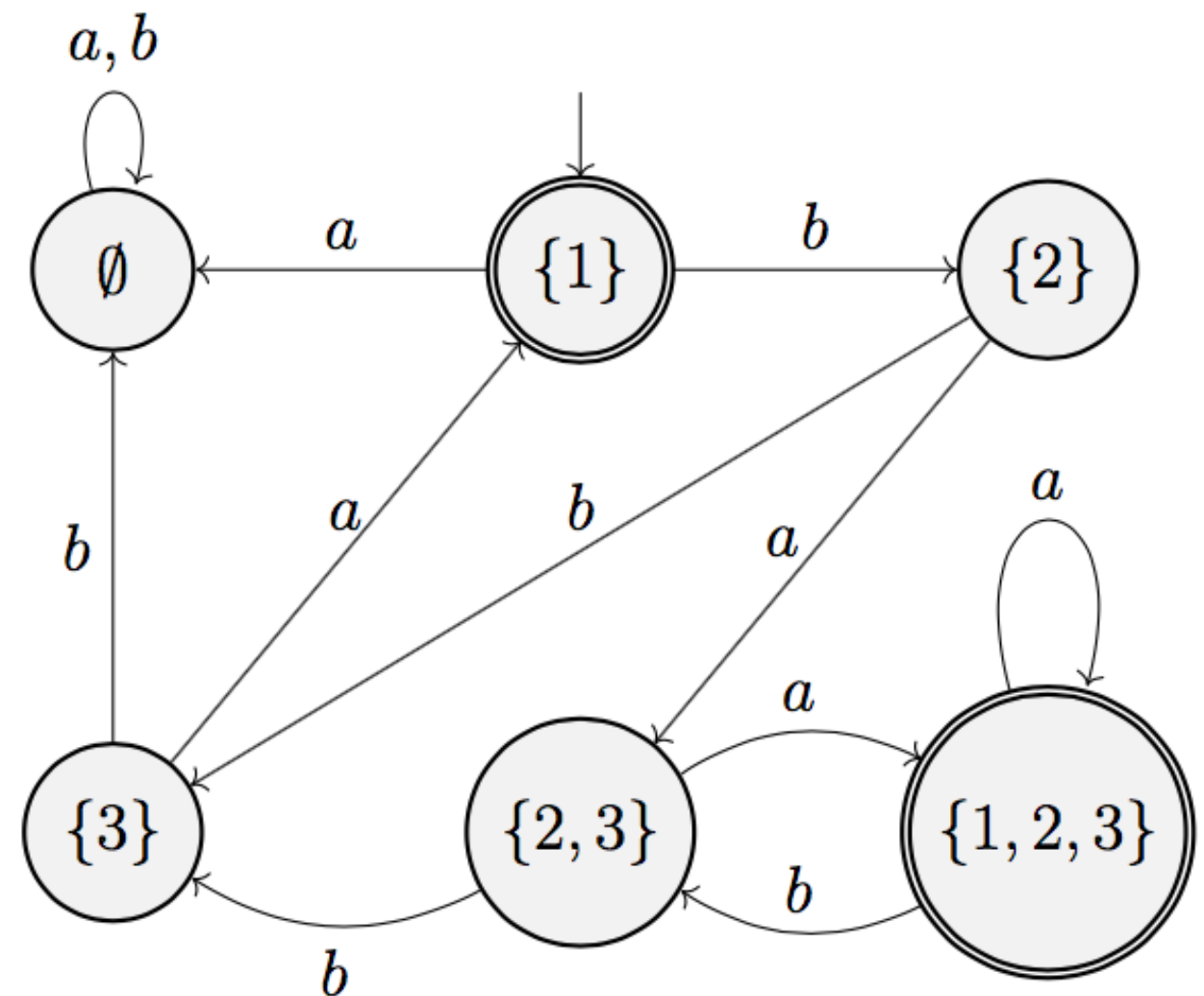
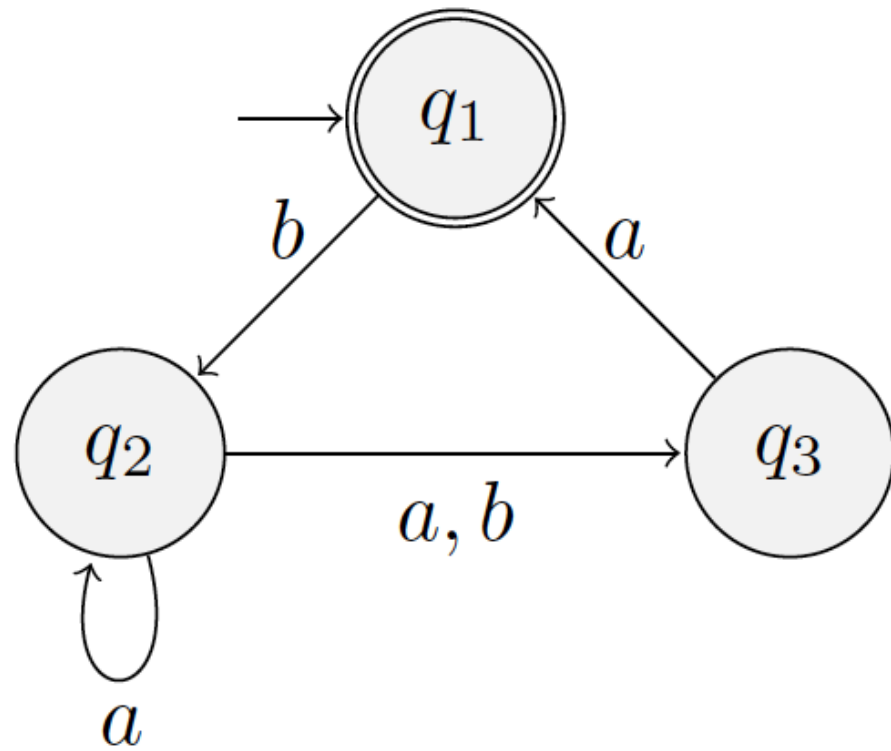
Example: Equivalent DFA?



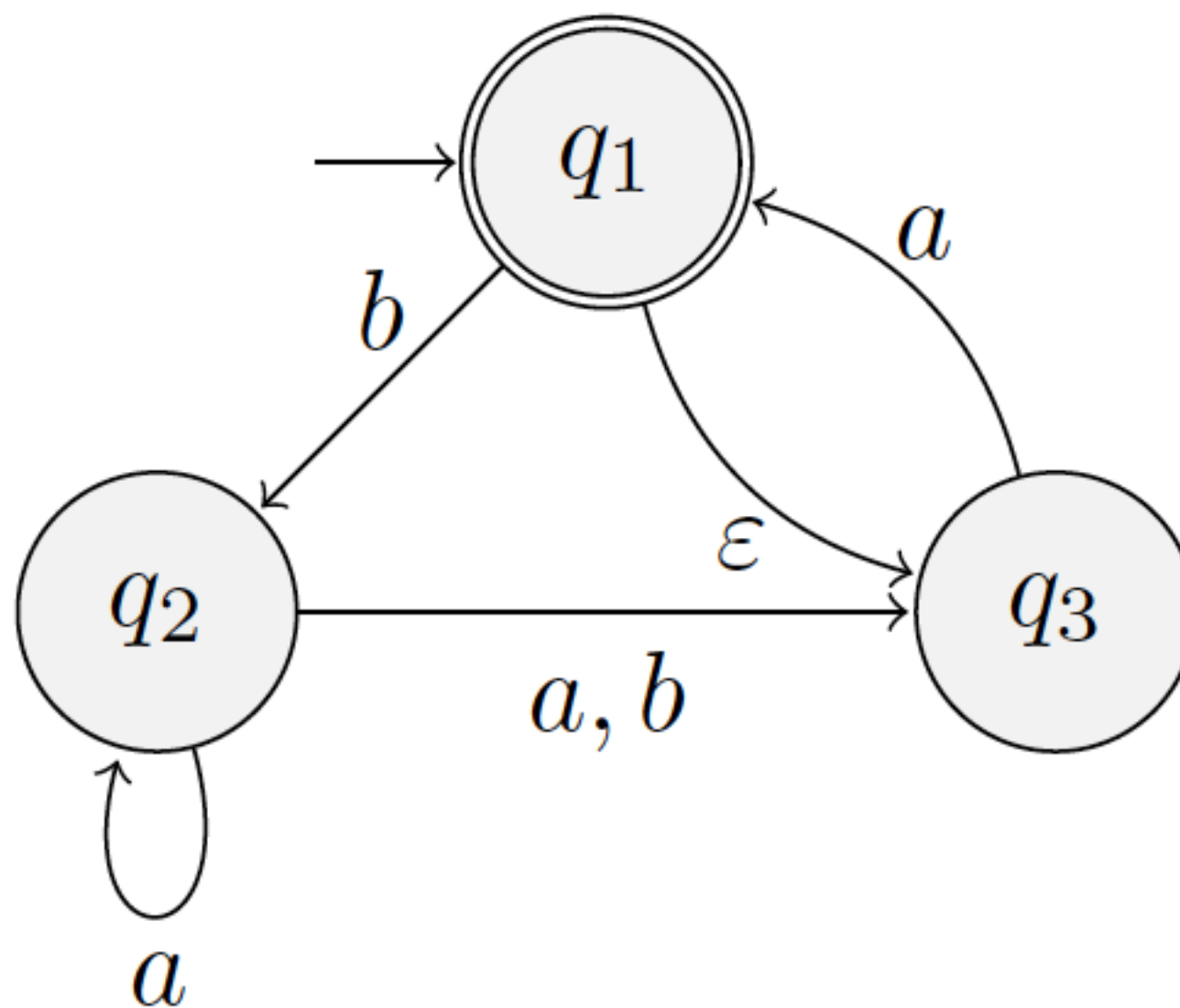
Creating an Equivalent DFA

- **Theorem.** Given any NFA $N = (Q, \Sigma, \delta, q, F)$ there exists an equivalent DFA M .
- **Proof.** $M = (Q_M, \Sigma, \delta_M, q_M, F_M)$ where
 - $Q_M = \mathcal{P}(Q)$
 - $q_M = \{q\}$
 - $\delta_M(R, a) = \cup_{q \in R} \delta(q, a)$ for any $R \in Q_M, a \in \Sigma$
 - $F_M = \{R \in Q_M \mid R \cap F \neq \emptyset\}$ (any "set" of states that contains an accept state of N)
- **Correctness:** $w \in L(N) \iff w \in L(M)$

Example: Equivalent DFA?



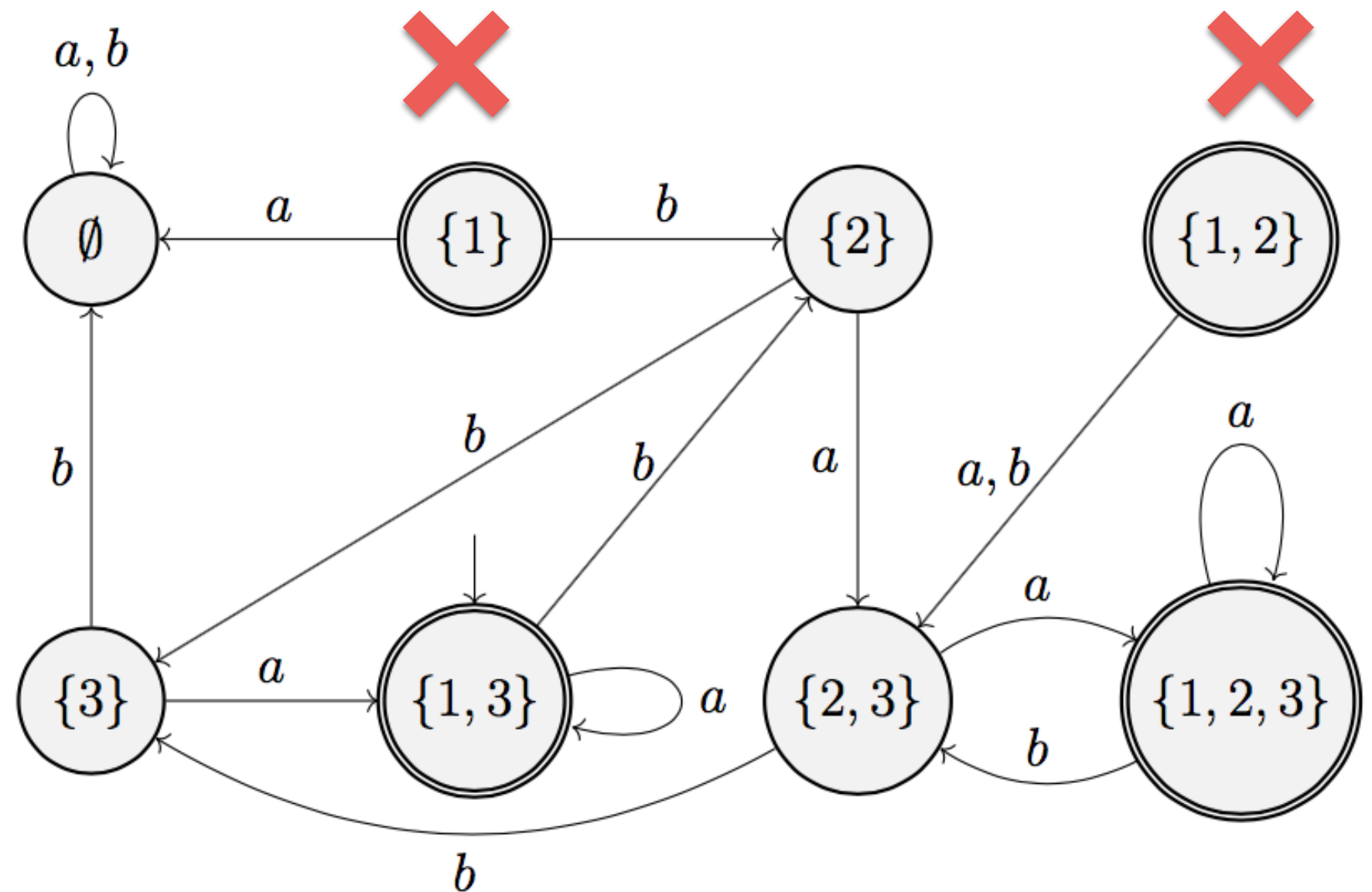
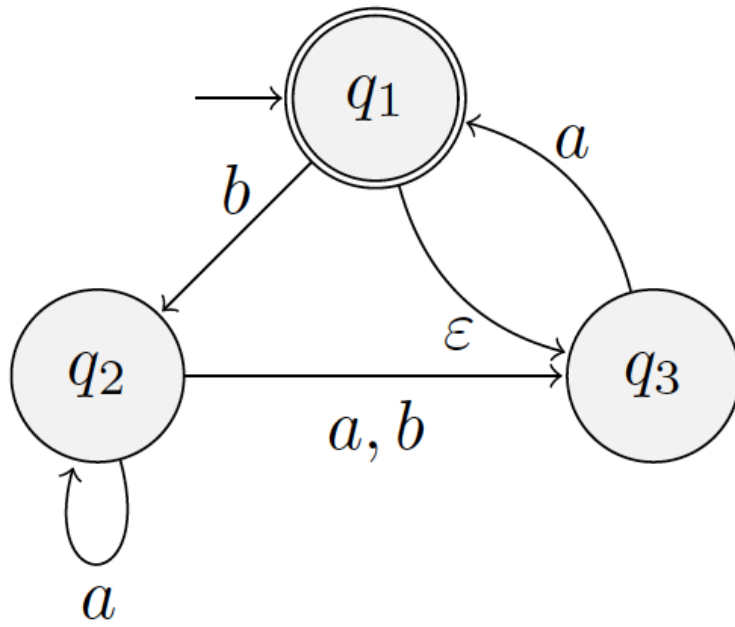
What about ε transitions?



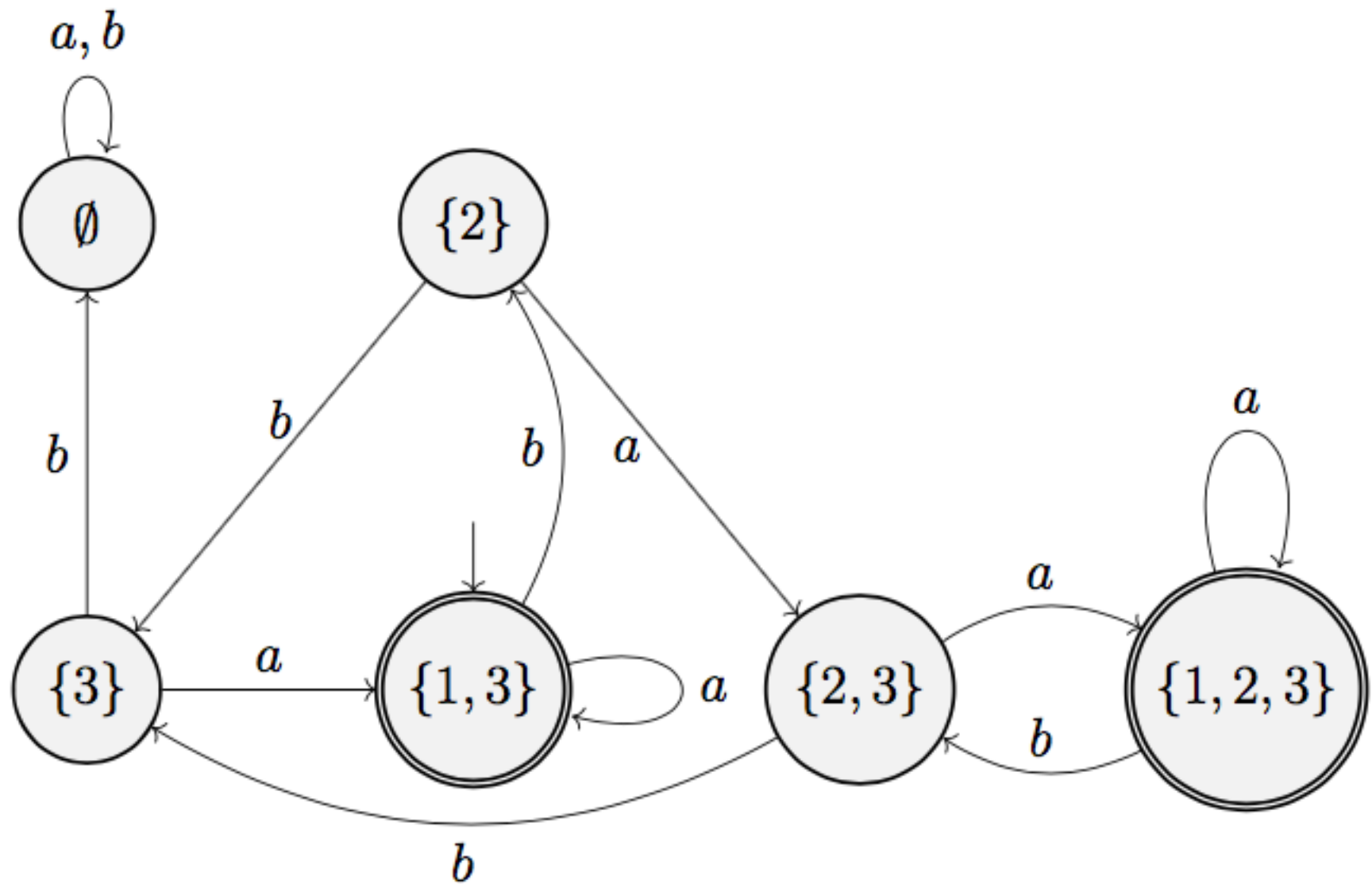
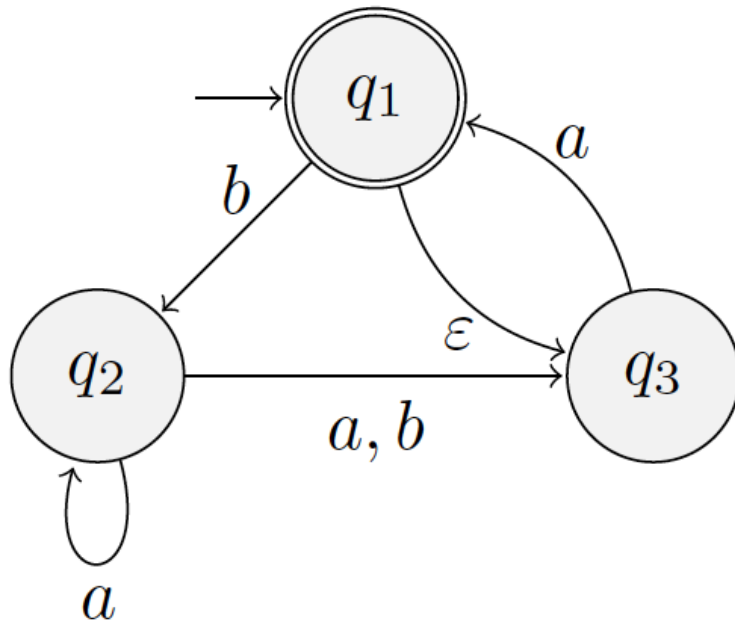
Creating an Equivalent DFA

- **Theorem.** Given any NFA $N = (Q, \Sigma, \delta, q, F)$ there exists an equivalent DFA M .
- **Proof.** $M = (Q_M, \Sigma, \delta_M, q_M, F_M)$ where $Q_M = \mathcal{P}(Q)$ and $F_M = \{R \in Q \mid R \cap F \neq \emptyset\}$ as before.
- **Definition.** (ε -closure) $E(R) = \{q \in Q \mid q \text{ can reached from any state in } R \text{ along zero or more } \varepsilon \text{ transitions} \}$
 - Notice that $R \subseteq E(R)$ and $E(R) \in Q_M$
- Now we can define the start state of M as: $q_M = E(\{q\})$
- Transition function $\delta(R, a) = \cup_{r \in R} E(\delta(r, a))$ for any $R \in Q_M, a \in \Sigma$

Equivalent DFA



Equivalent DFA

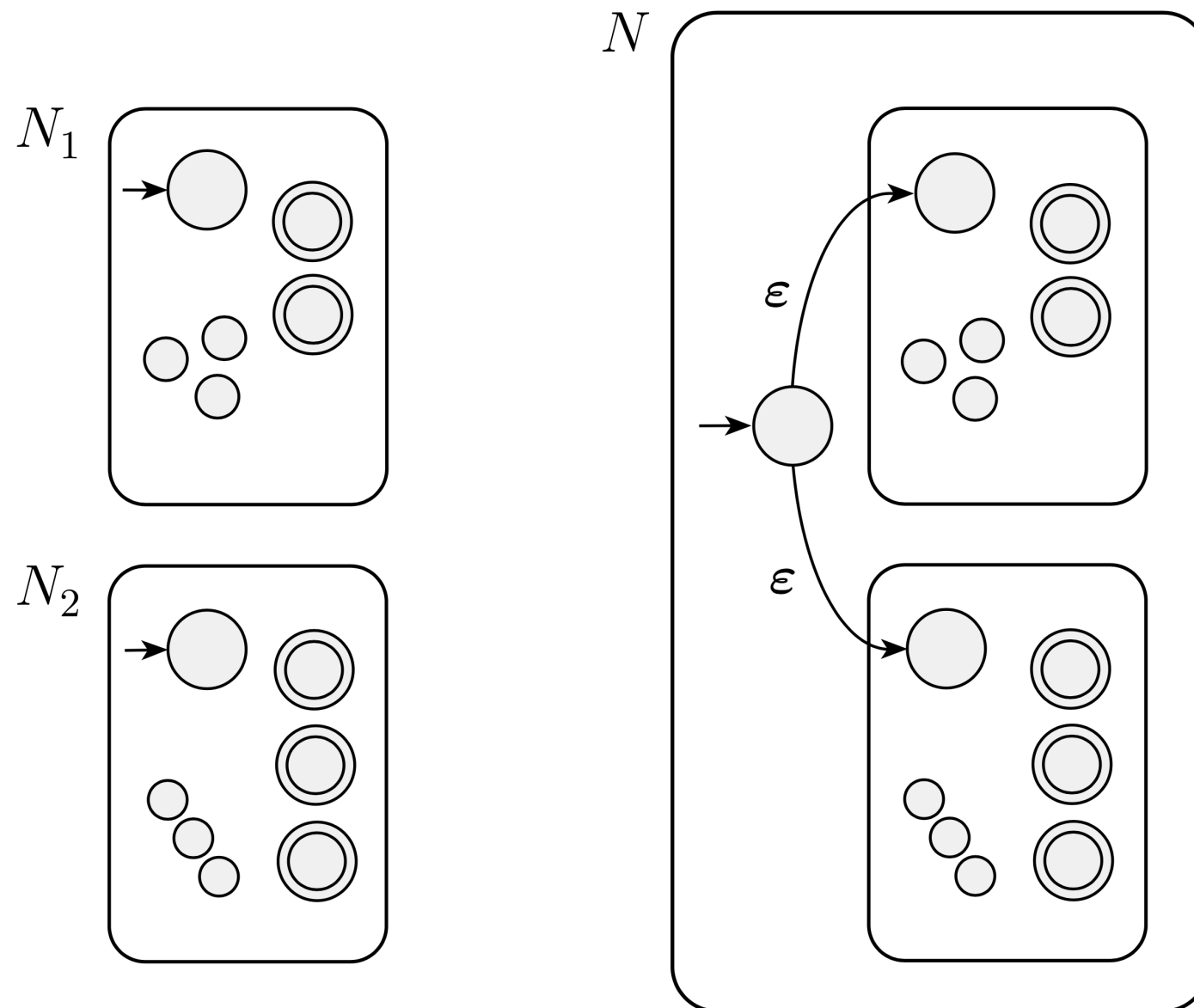


Alternate Definition of Regular Languages

- **Corollary.** A language is regular iff some NFA recognizes it.

Revisit Closure Under Union

- Let N_1 and N_2 be DFAs for languages L_1 and L_2
- **Question.** How to construct an NFA for $L_1 \cup L_2$?



Concatenation

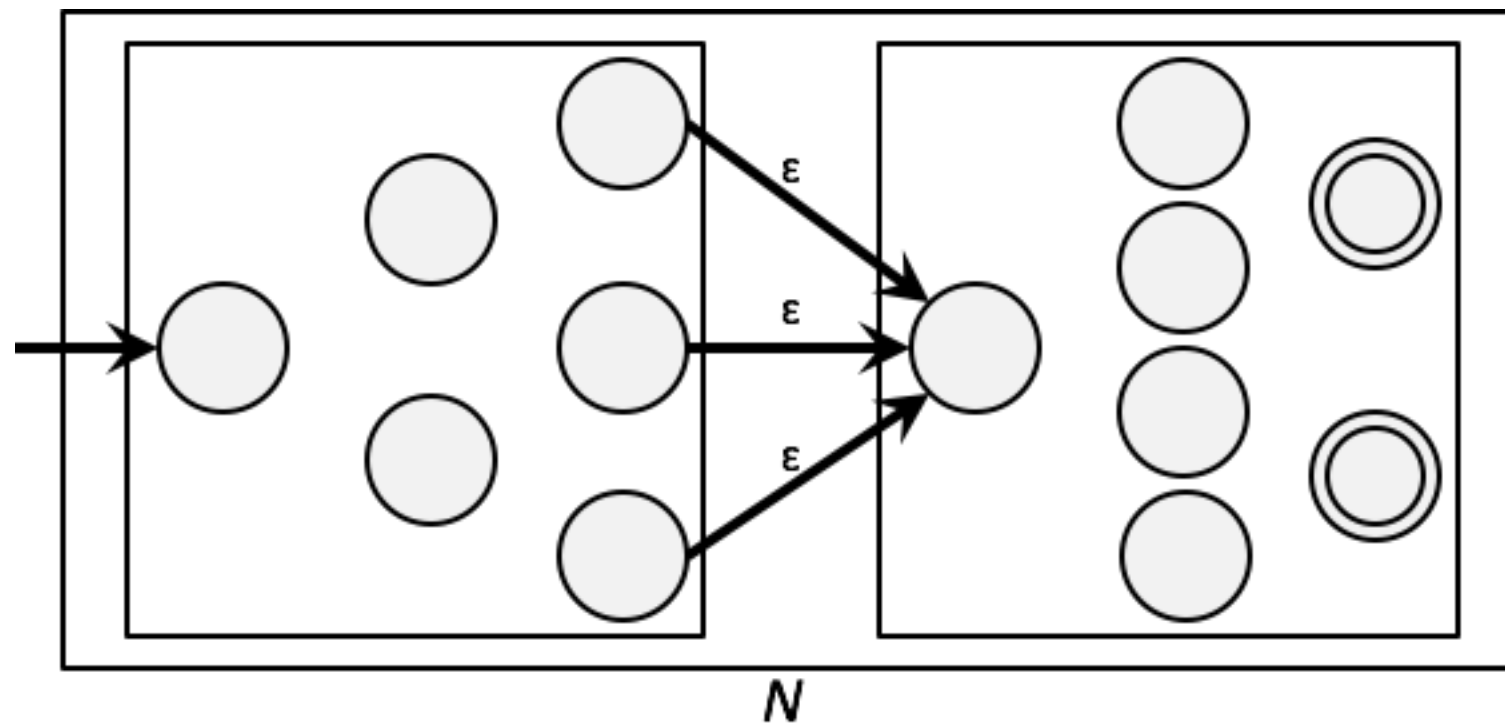
- Let A and B be languages over Σ .
- **Definition.** Concatenation of A and B , denoted $A \circ B$ is defined as

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

- **Theorem.** Regular languages are closed under concatenation.

Closed Under Concatenation

- **Theorem.** The class of languages are closed under concatenation.



Closed Under Concatenation

- **Theorem.** The class of languages are closed under concatenation.
- Proof. Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be the NFA for L_1 and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be the NFA for L_2
- Construct NFA $N = (Q, \Sigma, \delta, q_0, F)$ to recognize $L_1 \circ L_2$
 - $Q = Q_1 \cup Q_2$
 - $q_0 = q_1$
 - $F = F_2$
 - $\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2. \end{cases}$

Kleene Star

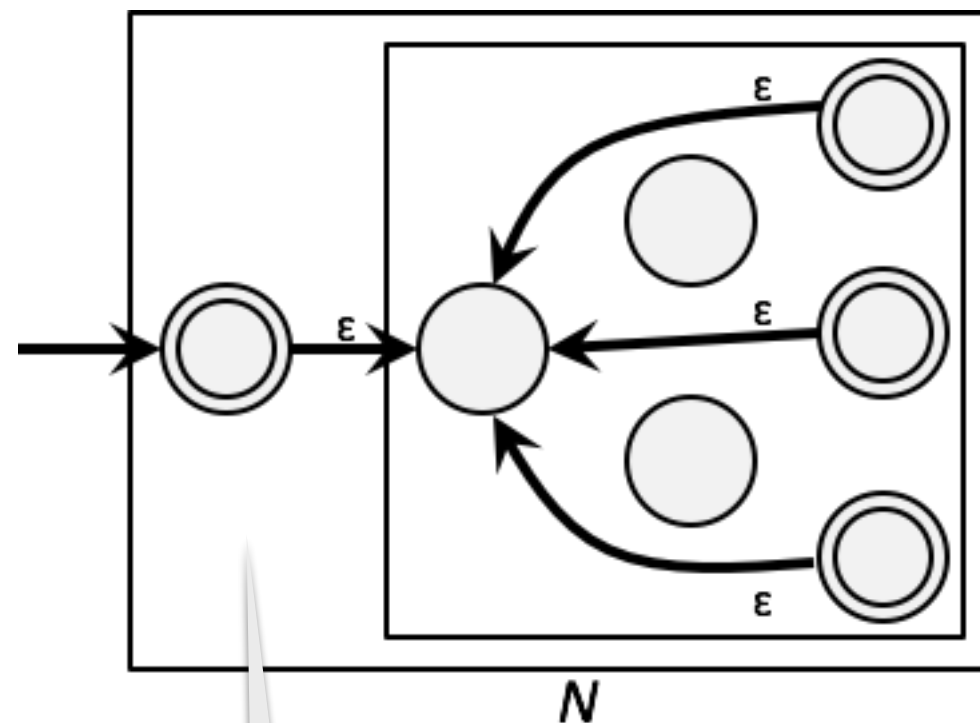
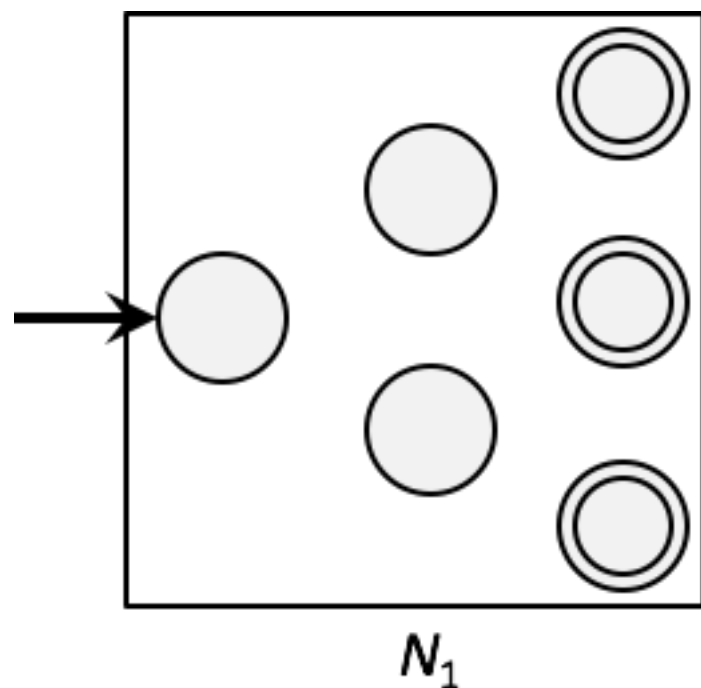
- Let A be a language on Σ
- Definition. Kleene star of A , denoted A^* is defined as:

$$A^* = \{w_1w_2\cdots w_k \mid k \geq 0 \text{ and each } w_i \in A\}$$

- **Example.** Suppose $L_1 = \{01,11\}$, what is L^* ?
- **Question.** Are regular languages closed under Kleene star?

Kleene Star

- **Theorem.** The class of regular languages is closed under Kleene star.



Do we need this new state? Why?

Closed Under Kleene Star

- **Theorem.** The class of languages are closed under Kleene star.
- Proof. Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be the NFA for L_1
- Construct NFA $N = (Q, \Sigma, \delta, q_0, F)$ to recognize L_1^*
 - $Q = Q_1 \cup \{q_0\}$ (add a new start state)
 - $F = F_1 \cup \{q_0\}$
 - $$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \epsilon \\ \{q_1\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & q = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

Not All Languages are Regular

- Intuition about regular languages:
 - DFA only has finitely many states, say k
 - Any string with at least k symbols: some DFA state is visited more than once
 - DFA "loops" on long enough strings
 - Can only recognize languages with such nice "regular" structure
- Will see general techniques for showing that a language is not regular
- Classic example of a language that is not regular:
 - $\{w = 0^n 1^n \mid n \geq 0\}$ (equal number of 0s and 1s)